

数学与算法补充 (附录 A F)

目录

线性代数与矩阵	3
A.1 直觉故事：矩阵到底在干什么	3
A.2 关键概念讲解：把名词翻译成通俗解释	3
A.3 深入：分解与算法（重点、难点）	4
A.4 动手例题（选做：手算 + 验证）	5
A.5 Python 对应（速查）	6
A.6 常见误区	6
A.7 使用章节（双向）	7
A.10 常见考题与自查（考前 10 分钟）	7
A.9 综合案例：SVD 图像压缩（串联全附录）	7
A.8 例题集（深入练习）	8
微积分与数值方法	8
B.1 直觉故事：从“瞬时速度”说起	8
B.2 关键概念（带手算小例子）	9
B.3 深入：数值方法（重点、难点）	9
B.4 动手例题（选做）	10
B.5 Python 对应（速查）	11
B.6 常见误区	11
B.7 使用章节（双向）	12
B.10 常见考题与自查（考前 10 分钟）	12
B.9 综合案例：拟合 + 积分（做一道完整数值题）	12
B.8 例题集（深入练习）	13
概率统计基础	13
C.1 直觉故事：为什么不能只看平均分	13
C.2 随机变量与分布（讲解）	13
C.3 统计推断（重点、难点）	14
C.4 回归与方差分析（建模视角）	15
C.5 时间序列：和“普通回归”有什么不同（重点）	15
C.6 动手例题（选做）	16
C.7 Python 对应（速查）	16
C.8 常见误区	17
C.9 使用章节（双向）	17
C.12 常见考题与自查（考前 10 分钟）	17
C.11 综合案例：一份“完整统计报告”的骨架	18
C.10 例题集（深入练习）	18

图论与网络	19
D.1 直觉故事: 朋友圈里的“谁最重要”	19
D.2 图的基本表示 (讲解 + 手算)	19
D.3 经典算法: 思路与手算	20
D.4 动手例题 (选做)	20
D.5 Python 对应 (速查)	21
D.6 常见误区	21
D.7 使用章节 (双向)	22
D.10 常见考题与自查 (考前 10 分钟)	22
D.9 综合案例: 给一个小网络做“体检”	22
D.8 例题集 (深入练习)	23
机器学习基础	23
E.1 直觉故事: 为什么模型会“学偏”	23
E.2 学习范式与损失函数 (讲解)	23
E.3 评估与过拟合 (重点、难点)	24
E.4 常用模型直觉 (讲解 + 速查)	24
E.5 动手例题 (选做)	25
E.6 Python 对应 (scikit-learn 标准流程, 速查)	25
E.7 常见误区	26
E.8 使用章节 (双向)	26
E.11 常见考题与自查 (考前 10 分钟)	26
E.10 综合案例: 一次完整的建模流程 (期末大作业可直接套)	27
E.9 例题集 (深入练习)	27
通用数学与算法速查	28
F.1 常用记号与希腊字母 (速查)	28
F.2 常用公式卡 (速查)	28
F.3 复杂度与常用算法量级 (速查)	29
F.4 数值稳定性 (科学计算必修, 讲解)	29
F.5 关键词索引 (速查)	29
F.6 使用章节 (双向)	30
F.7 术语英中对照 (速查)	30
F.9 考前 10 分钟清单 (全附录)	31
F.8 数值小技巧 (速查)	31
全书总参考文献 (附录 G)	31
一、官方文档与工具链	31
二、教材与书籍	34
三、精品教程与课程	35
四、习题与实战	36
五、开源项目与配套仓库	37
六、中文资料与镜像	37
七、其他	37
按章节快速查找	38

线性代数与矩阵

本附录讲科学计算里最常用的线性代数：向量/矩阵的直觉、关键定义、分解与算法（重点是特征值与 SVD）、数值稳定性，以及 Python 对应。学完你应能用“线性变换”的眼光看第 1 章 NumPy 的线性代数与第 8 章 sklearn 的降维。

A.1 直觉故事：矩阵到底在干什么

很多同学背熟了矩阵乘法，却不知道它在“算什么”。从一个具体问题开始：

一家店卖两种套餐：A 套餐（米饭 1 份 + 鸡肉 2 份），B 套餐（米饭 2 份 + 鸡肉 1 份）。卖出 3 份 A、2 份 B，一共用了多少米饭和鸡肉？

卖出数量写成向量 $x = (3, 2)$ ；A、B 套餐用料写成列向量 $(1, 2)$ 与 $(2, 1)$ 。总用料就是**线性组合**：

米饭 = $3 \times 1 + 2 \times 2 = 7$, 鸡肉 = $3 \times 2 + 2 \times 1 = 8$

写成矩阵：

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

这个例子说明三件事：

- 1. **向量是“配比”**：一行数据、一份订单、一个状态，都可以是一个向量；
- 2. **矩阵是“线性变换的约定”**：A 的每一列是一种“原料配方”，Ax 就是按 x 的比例把各列混合起来；
- 3. **矩阵乘法 = 复合变换**：先做 B 再做 A，顺序不能换——所以一般 $AB \neq BA$ 。

正文见：1 NumPy ·03 线性代数、1 NumPy ·04 多项式与拟合。

A.2 关键概念讲解：把名词翻译成通俗解释

A.2.1 转置、内积与范数

概念	定义	通俗解释
转置	A^T	行列互换； $(AB)^T = B^T A^T$
内积	$x \cdot y = x^T y$	两个向量有多“同向”
范数	$\ x\ _2 = \sqrt{x^T x}$	向量长度

例子： $x = (1, 2)$, $y = (3, 4)$, 则 $x \cdot y = 1 \times 3 + 2 \times 4 = 11$, $\|x\|_2 = \sqrt{5} \approx 2.24$ 。内积大且同向 → 相似度高：这是余弦相似度、皮尔逊相关、注意力机制的底层直觉。

A.2.2 逆、行列式与秩：能否把变换“倒回去”

- **逆 A^{-1}** ：把变换倒着做一遍， $A^{-1} A = I$ ；

- **行列式** $\det(A)$: 变换后“体积”放大的倍数; $\det(A) = 0$ 表示被压扁 (不可逆);
- **秩** $\text{rank}(A)$: 变换后空间的真正维度 (列向量中线性无关的个数); 满秩方阵才可逆。

例子: $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ 。 $\det(A) = 4 - 1 = 3 \neq 0$, 可逆; $A^{-1} = \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ 。而 $B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ 两列成比例, $\text{rank}(B) = 1$, $\det(B) = 0$ ——它把平面压成一条线, $Bx = b$ 要么无解要么无穷多解。

A.2.3 特征值与特征向量: 变换的“不动轴”

定义: 若 $Av = \lambda v$ ($v \neq 0$), 则 v 是特征向量, λ 是特征值。通俗解释: **有的方向被矩阵只“拉伸”不“拐弯”, 拉伸倍数是特征值。**

对 A 手算: $\det(A - \lambda I) = (A - \lambda I)$ 展开得 $\lambda_1 = 3$ (方向 $(1, 1)$)、 $\lambda_2 = 1$ (方向 $(1, -1)$)。即: 沿对角线方向放 3 倍、沿反对角线方向不动。

用途: - **对称矩阵**: $A = Q\Lambda Q^T$ (Q 正交), 坐标轴转成“特征方向”后问题立刻解耦——PCA 就是找这些方向; - **幂迭代 / PageRank / 马尔可夫链稳态**: 都在找最大特征值对应的特征向量。

手算展开式: $\begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = (2-\lambda)^2 - 1 = 0$, 即 $\lambda = 1$ 或 3 。

A.2.4 条件数: 解有多“敏感”

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

直觉: b 变化一点点, x 可能变化很多倍。条件数大 (病态) $\neq$ 无解, 而是**数值上很难算准**, 做模型/拟合前值得看一眼。

正文见: [1 NumPy ·03 线性代数](#) (条件数/病态)、[3 SciPy ·02 优化工具包](#)。

A.3 深入: 分解与算法 (重点、难点)

A.3.1 解方程组: 高斯消元与 LU

求解 $Ax = b$ 最直接是高斯消元 (上三角 + 回代)。消元等价于 $A = LU$: **一次分解、多次求解**, 解多组 b 时很划算。

- 复杂度 $O(n^3)$; 必须**部分主元** (选最大元素当主元), 避免小除数放大误差;
- 代码: `np.linalg.solve` (带主元的 LU), **不要用** `np.linalg.inv(A) @ b`。

A.3.2 特征值分解: 对称矩阵为什么“好”

- 一般方阵: $A = V\Lambda V^{-1}$ (可对角化时)。特征向量可能不正交, 且可能是复数;
- **对称矩阵**: $A = A^T$ 时特征值全是实数, 特征向量可正交归一: $A = Q\Lambda Q^T$ 。此时 $A^k = Q\Lambda^k Q^T$, 求幂、求逆、判断正定都极简单;
- 用途: 二次型分析 ($x^T A x$ 的正定性 $\Rightarrow$ 凸性)、PCA、谱聚类、稳定性分析。

A.3.3 SVD: 最重要的一段 (本科必会)

定理 (SVD): 任意 $m \times n$ 矩阵 A 可写成

$$A = U \Sigma V^T$$

其中 U ($m \times m$)、 V ($n \times n$) 是正交矩阵, Σ 是 $m \times n$ 的“ 对角矩阵”, 对角线元素 $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ 叫**奇异值**。

几何解读 (三步): 先旋转 (V^T) $\rightarrow$ 逐方向缩放 (Σ) $\rightarrow$ 再旋转 (U)。奇异值 = 每个方向的“ 重要性/能量”。

与特征值的关系: $A^T A$ 的特征值是 σ_i^2 , V 是 $A^T A$ 的特征向量; AA^T 给出 U 。所以 SVD “ 任意矩阵都能做”, 是特征分解的“ 万能推广”。

三个用途 (考试/作业高频):

1. **最优低秩近似** (Eckart–Young): 保留前 k 大奇异值得到 A_k , 是所有秩 k 矩阵中与 A 的 Frobenius 距离最小者。

$$\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2$$

——图像压缩、去噪、推荐系统都靠它。

2. **PCA**: 对数据中心化后的矩阵做 SVD, V 的前 k 列就是主成分方向; 奇异值平方占比 = 解释方差比。
3. **伪逆与最小二乘**: $A^+ = V \Sigma^{-1} U^T$ (Σ^{-1} 取非零奇异值的倒数), 比正规方程 $(A^T A)^{-1} A^T b$ 数值更稳 (避免把条件数平方)。

数值上怎么算: 实际用 QR 迭代/分治法 (库内实现), 复杂度约 $O(mn^2)$ (薄 SVD)。代码只需 `np.linalg.svd`。

A.3.4 最小二乘与正则化

超定方程组 $Ax \approx b$ 的最小二乘解:

$$x^* = \arg \min_x \|Ax - b\|_2^2 \implies A^T Ax = A^T b$$

- 几何: 把 b 投影到 A 的列空间, 误差向量与列空间垂直;
- 数值: 优先 A^+b (SVD) 或 QR; 直接求 $(A^T A)^{-1}$ 会平方条件数;
- 加惩罚 $\lambda \|x\|^2$ (岭回归): 在“ 拟合” 与“ 不放大” 之间取舍——这就是第 8 章正则化的数学原型。

A.3.5 数值稳定性: 本科最易踩的坑

坑	例子	正确做法
灾难性消去	$\sqrt{x^2 + 1} - x$ (大 x)	改写 $1/(\sqrt{x^2 + 1} + x)$
显式求逆	$A^{-1}b$	$Ax = b$ 用 <code>np.linalg.solve</code>
病态不检查	条件数 10^{12} 硬解	先看 <code>cond(A)</code> , 换模型/加正则
忘记缩放	PCA/距离计算被大数值特征主导	先标准化

A.4 动手例题 (选做: 手算 + 验证)

例 1: 解方程组. $2x + y = 1, x + 2y = 2$. 代入得 $x = 0, y = 1$. 验证: `np.linalg.solve` 得 $[0, 1]$ 。

例 2: SVD 手算 2×2。 $A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ 。 $A^T A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$ ，奇异值 $\sigma_1 = \sigma_2 = \sqrt{2}$ ； $V = I$, $U = \frac{1}{\sqrt{2}}A$ 。
这说明对称正交型矩阵“只缩放不旋转”，是 SVD 最简单的例子。

```
import numpy as np
A = np.array([[1., 1.], [1., -1.]])
U, S, Vt = np.linalg.svd(A, full_matrices=False)
print(S, U @ np.diag(S) @ Vt)      # 还原 A
```

A.5 Python 对应（速查）

```
import numpy as np, sympy as sp
A = np.array([[2, 1], [1, 2]]); b = np.array([[1], [2]])

np.linalg.solve(A, b)                # 解方程组
np.linalg.det(A); np.linalg.matrix_rank(A)
np.linalg.eig(A)                      # 特征值/特征向量
np.linalg.eigh(A) if (A == A.T).all() else None
np.linalg.svd(A, full_matrices=False) # U, S, Vt (S 是向量!)
np.linalg.lstsq(A, b, rcond=None)
np.linalg.cond(A); np.linalg.pinv(A)
sp.Matrix(A).eigenvecs()              # SymPy 符号版
```

你想要	用哪个
解方阵方程组	np.linalg.solve
最小二乘	np.linalg.lstsq
特征值（对称）	np.linalg.eigh
SVD	np.linalg.svd (注意 S 是向量)
判断病态	np.linalg.cond

A.6 常见误区

误区	正确
A * B 当矩阵乘	* 是逐元素；矩阵乘用 A @ B
用 inv(A) @ b	用 solve / lstsq
以为 S 是矩阵	SVD 返回奇异值向量，需 np.diag(S)
忽略条件数	病态时解会“抖”，先看 cond
非对称矩阵用 eigh	eigh 要求对称；否则用 eig
以为 Σ 全为正	非负奇异值；若出现 0 说明有“压扁”方向（降秩）

A.7 使用章节（双向）

章	哪里用到	链接
1 NumPy	03 线性代数、05 SVD 压缩	正文
2 SymPy	矩阵与特征值	正文
3 SciPy	优化/插值中的矩阵	正文
8 sklearn	PCA/降维	正文

下游衔接：intro-mathmodel 第 1 章（Numpy 与线性代数）、第 3 章（线性规划 → 非线性规划）。**延伸阅读：**mml-book 第 2~4 章、SciPy Lectures、numpy.linalg 官方文档（见 [references.md](#)）。

A.10 常见考题与自查（考前 10 分钟）

会了吗？	考点	一句话答案
☐	det=0 意味着什么	列线性相关，变换压扁，不可逆
☐	特征向量的几何含义	只被拉伸不改方向，拉伸倍数是特征值
☐	SVD 与 PCA 的关系	中心化后 SVD 的右奇异向量 = 主成分方向
☐	条件数干什么	衡量解对输入扰动的敏感度
☐	最小二乘的几何	把 b 投影到列空间，残差垂直列空间
☐	为什么不用 $(A^T A)^{-1}$	会平方条件数，数值不稳
☐	手算 2×2 特征值	特征多项式 $\det(A - \lambda I) = 0$

A.9 综合案例：SVD 图像压缩（串联全附录）

目标：把一张灰度图当作矩阵 A，用 SVD 保留前 k 个奇异值，观察压缩率与重构误差。

完整流程：

1. 读灰度图 → 数值矩阵 A (m 行 n 列)；
2. 做薄 SVD: $A = U S V^T$ (S 是奇异值向量)；
3. 取前 k 个: $A_k = U[:, :k] @ \text{diag}(S[:k]) @ V^T[k, :]$ ；
4. 比较：存储量从 mn 降到 k(m+n)；误差 = 被丢弃奇异值的平方和。

手算小例：2×2 矩阵 $A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ ，奇异值都是 $\sqrt{2}$ 。若保留 $k=1$ ： $A_1 = U_1 * \sqrt{2} * V_1^T$ ，会丢掉“反对角信息”——对这张图来说信息全无，说明“前 k 个”要按奇异值落差选。

参考要点代码（四空格缩进可直接运行）：

```
import numpy as np, matplotlib.pyplot as plt
A = np.random.default_rng(0).random((60, 60))
U, S, Vt = np.linalg.svd(A, full_matrices=False)
for k in [5, 15, 30]:
    Ak = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
    mse = np.mean((A - Ak) ** 2)
```

```
ratio = A.size / (k * (A.shape[0] + A.shape[1]))
print(k, "MSE:", round(mse, 4), "压缩比:", round(ratio, 1))
```

反思：SVD 是“数据驱动压缩”的数学核心；学完写一句结论：“奇异值大的方向 = 主要信息；保留它们的低秩近似 = 最优压缩。”

A.8 例题集（深入练习）

例 1：特征值代入法。 $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ ，特征多项式 $\det(A - \lambda I) = \begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = (2-\lambda)^2 - 1 = 0$ ，得 $\lambda = 1, 3$ 。代入 $\lambda = 3$ ： $\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} v = 0 \rightarrow v = (1, 1)$ 。这就是“先求特征值、再解齐次方程组”的标准流程。

例 2：条件数实验（为什么不能直接求逆）。 构造接近奇异的 $A = \begin{bmatrix} 1 & 1 \\ 1 & 1.001 \end{bmatrix}$ ，条件数约 4000： b 稍微变动，解差很大。代码：

```
import numpy as np
A = np.array([[1., 1.], [1., 1.001]])
print(np.linalg.cond(A))           # 约 4000 (病态)
b1 = np.array([1., 1.]); b2 = np.array([1., 1.00001])
print(np.linalg.solve(A, b1), np.linalg.solve(A, b2)) # 解差异大
```

例 3：SVD 压缩手算。 对 $A = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$ ：奇异值 1, 1，任意正交 U, V 。去掉一个奇异值得到零矩阵——说明“信息量全在奇异值里”。

微积分与数值方法

本附录为第 2 章 SymPy（符号推导）与第 3 章 SciPy（数值工具）提供数学背景：微积分直觉、关键公式、数值方法（积分/优化/插值/FFT/ODE）与本科常见的难点（误差阶、收敛性、刚性问题）。

B.1 直觉故事：从“瞬时速度”说起

开车两小时走 180 公里，平均 90 km/h；但“先快后慢”时，平均速度看不出细节。

- **导数** = 瞬时变化率：时间间隔缩到无限小，平均速度的极限。几何 = 切线斜率；机器学习里 = “参数往哪调、调多少”。
- **积分** = 累积量：已知每秒速度求总路程 = 把速度乘时间加总，间隔越细越准。几何 = 曲线下面积。
- **泰勒展开**： $f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \dots$ ——把非线性函数在局部“线性化/抛物化”。牛顿法、误差分析、反向传播都靠它。

正文见： [2 SymPy ·01 符号对象与基本运算](#)、[2 SymPy ·02 利用 SymPy 求解问题](#)。

B.2 关键概念 (带手算小例子)

B.2.1 导数、偏导与梯度

手算 $f(x) = x^2$ 在 $x = 2$: $\frac{(2+h)^2 - 4}{h} = 4 + h \rightarrow 4$ 。所以 $f'(2) = 4$ 。

多元 $\rightarrow$ 偏导 $\rightarrow$ 梯度:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)$$

含义: 指向 f 上升最快的方向; 负梯度 = 最速下降方向。

导数速查: $(x^n)' = nx^{n-1}$ 、 $(\sin x)' = \cos x$ 、 $(e^x)' = e^x$ 、 $(\ln x)' = 1/x$ 、 $(uv)' = u'v + uv'$ 、 $(f(g(x)))' = f'(g(x))g'(x)$ 。

B.2.2 积分与基本定理

- 不定积分 = 求原函数; 定积分 = 面积/累积;
- 基本定理: $\int_a^b f'(x) dx = f(b) - f(a)$;
- 手算: $\int_0^1 3x^2 dx = x^3 \Big|_0^1 = 1$ 。

B.2.3 为什么还要“数值” (难点)

实际函数常无解析式, 或表达式复杂到算不动。近似方法:

- **数值微分** (中心差分): $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$, 误差 $O(h^2)$ (比前向差分 $O(h)$ 好);
- **数值积分**: 梯形 (一阶)、辛普森 (二阶抛物线)、高斯求积 (高阶、自适应加密)。**误差阶告诉你“步长减半, 误差怎么降”——这是数值分析最核心的思想。**

B.3 深入: 数值方法 (重点、难点)

B.3.1 优化: 找“最低点”

方法	更新式	记忆点
梯度下降	$x \leftarrow x - \alpha \nabla f(x)$	沿最陡下坡走; 步长要调
牛顿法	$x \leftarrow x - H^{-1} \nabla f(x)$	用二阶曲率加速
拟牛顿/BFGS	近似 H^{-1}	<code>scipy.optimize.minimize</code> 默认
约束优化	KKT 条件	带 $g(x) \leq 0$

为什么牛顿法快: f 在 x_k 附近用二阶泰勒近似, 令导数为零得 $x_{k+1} = x_k - H^{-1} \nabla f$ 。凸、光滑、好初值时二次收敛 (误差平方级下降)。

步长手算： $f(x) = (x - 3)^2$, $x_0 = 1$, $\nabla f = 2(x - 3) = -4$ 。 $\alpha = 0.5 \rightarrow$ 一步到 $x = 3$; $\alpha = 2 \rightarrow x = 9$, 直接弹过最优——“步长太大震荡”。

B.3.2 插值 vs 拟合（本科高频考点）

- **插值：**曲线**穿过**每个已知点（拉格朗日、样条）；高次会**龙格震荡**（边界剧烈摆动）——不要迷信“高次”；
- **拟合：**曲线**接近**所有点（最小二乘，见附录 A）；噪声数据永远优先拟合（+ 正则化）；
- **选择：**数据精确且要过点 $\rightarrow$ 样条插值；有噪声 $\rightarrow$ 拟合。

B.3.3 傅里叶变换（FFT）

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-2\pi i k n / N}$$

把信号拆成不同频率正弦叠加。FFT 复杂度 $O(N \log N)$ （直接 DFT 是 $O(N^2)$ ）；用途：找周期、去噪、平滑；截断信号会**频谱泄漏**（加窗缓解）。

B.3.4 常微分方程（ODE）

初值问题 $y'(t) = f(t, y)$, $y(t_0) = y_0$:

- **欧拉法：** $y_{n+1} = y_n + hf(t_n, y_n)$ ——简单但误差大 ($O(h)$)；
- **RK45：**每步取 4~5 个点的加权平均，误差约 $O(h^4)$, `scipy.integrate.solve_ivp` 默认；
- **刚性问题：**特征值相差悬殊时，显式方法步长被迫极小 $\rightarrow$ 换隐式方法 `method = "Radau"`。

B.3.5 怎么选数值方法（决策表）

场景	首选	理由
有解析式、要精确结构	SymPy	数值只是近似
只有采样数据、要面积	quad	自适应、精度高
求极值、无导数	minimize (BFGS)	稳健默认
数据精确要过点	样条插值	高次插值震荡
数据有噪声	最小二乘	插值会“记住”噪声
找周期/滤波	FFT	时域卷积长序列慢
ODE 初值	solve_ivp (RK45)	精度/效率均衡；刚性用 Radau

B.4 动手例题（选做）

例 1：数值积分 vs 解析。 $\int_0^1 3x^2 dx = 1$ ；梯形给 0.75，辛普森给 1.0。

例 2：中心差分。 $f(x) = x^2$, $h = 0.1$: $(f(2.1) - f(1.9))/0.2 = (4.41 - 3.61)/0.2 = 4.0$ （真值 $f'(2) = 4$ ）。

```
from scipy import integrate, optimize
import numpy as np
```

```
print(integrate.quad(lambda x: 3*x**2, 0, 1)[0])           # 1.0
print(optimize.minimize(lambda x: (x[0]-3)**2, x0=[1.]).x)  # ~3
x = np.array([1.9, 2.0, 2.1])
print(np.gradient(x*x, 0.1)[1])                          # ~4.0
```

B.5 Python 对应（速查）

```
import numpy as np, sympy as sp
from scipy import integrate, optimize, interpolate, fft

x = sp.symbols("x")
sp.diff(sp.sin(x), x); sp.integrate(3*x**2, (x, 0, 1))
integrate.quad(np.sin, 0, np.pi)
optimize.minimize(lambda x: (x[0]-2)**2+(x[1]+1)**2, x0=[0, 0])
interpolate.interpld([0,1,2], [0,1,4], kind="cubic")
np.polyfit([0,1,2], [0,1,4], 2)
fft.fft(signal)
integrate.solve_ivp(lambda t, y: -y, [0, 5], [1.0])
```

你想要	用哪个
符号导数/积分	sp.diff / sp.integrate
数值积分	scipy.integrate.quad
数值微分	np.gradient
优化	scipy.optimize.minimize
插值/拟合	interpolate.interpld / np.polyfit
FFT	scipy.fft.fft
ODE	solve_ivp

B.6 常见误区

误区	正确
插值 = 拟合	插值过点、拟合逼近；有噪声用拟合
优化不设初值	非凸问题初值决定结果；多试几组
数值微分直接用于噪声数据	先平滑/拟合再求导
FFT 只能处理声音	任何时序都能做频谱
刚性系统用默认求解器	换 method="Radau"
只看 $O(h)$ 就选方法	还要看常数、稳定性与实现难度

B.7 使用章节（双向）

章	哪里用到	链接
2 SymPy	极限/导数/积分/方程/ODE	01 符号对象与基本运算
3 SciPy	积分/优化/插值/FFT/ODE	01 微积分工具包
1 NumPy	多项式拟合	04 多项式与拟合

下游衔接：intro-mathmodel 第 2 章（微分方程与动力系统）、第 3 章（函数极值与规划模型）、第 6 章（数据处理与拟合模型）。**延伸阅读：**SciPy Lectures（优化/积分）、MIT OCW 数值计算、《数值分析》教材（见[references.md](#)）。

B.10 常见考题与自查（考前 10 分钟）

会了吗？	考点	一句话答案
☐	中心差分为什么优于前向	误差 $O(h^2)$ 而非 $O(h)$
☐	牛顿法为什么快	用二阶泰勒，凸光滑好初值二次收敛
☐	插值和拟合的区别	插值过点、拟合逼近；有噪声用拟合
☐	FFT 复杂度	$O(N \log N)$
☐	欧拉法 vs RK45	误差 $O(h)$ vs $O(h^4)$ ，RK 更稳更准
☐	优化步长太大	震荡不收敛；太小则慢
☐	刚性系统	换隐式方法（Radau）

B.9 综合案例：拟合 + 积分（做一道完整数值题）

问题：已知某产品的“价格-销量”散点，希望给出“总需求”（曲线下的面积）与“最优定价”（利润最大点）。

步骤：

1. 用最小二乘拟合价格 p 与销量 q 的线性/二次关系（附录 A 的正规方程或 `np.polyfit`）；
2. 用数值积分估算某区间的“总销量”（面积）；
3. 用优化求利润 $f(p) = p \cdot q(p)$ 的最大值点；
4. 用中心差分验证最优点的导数为 0。

参考要点代码：

```
import numpy as np
from scipy import integrate, optimize
p = np.array([1., 2., 3., 4.]); q = np.array([9., 7., 5., 3.])
coef = np.polyfit(p, q, 2)
total = integrate.quad(lambda x: np.polyval(coef, x), 1, 4)[0]
profit = lambda x: -x * np.polyval(coef, x)
best = optimize.minimize_scalar(profit, bounds=(1, 4), method="bounded")
print(total, best.x)
```

反思：把“数据 → 模型 → 积分 → 优化”串起来，就是科学计算的常用闭环。

B.8 例题集 (深入练习)

例 1：牛顿法手算两步。 $f(x) = x^2 - 2$, 求 $\sqrt{2}$. $f'(x) = 2x$, $x_{k+1} = x_k - \frac{x_k^2 - 2}{2x_k}$. 从 $x_0 = 1$: $x_1 = 1.5$, $x_2 = 1.4167$, $x_3 = 1.4142$ ——收敛到 $\sqrt{2} \approx 1.41421356$ (二次收敛)。

例 2：误差阶实验。用梯形法则算 $\int_0^1 x^2 dx = 1/3$: $h = 0.5$ 得 0.375, $h = 0.25$ 得 0.34375。步长减半, 误差从 0.0417 降到 0.0104——大约除以 4 (二阶收敛)。

```
from scipy import integrate
for h in [0.5, 0.25]:
    xs = np.arange(0, 1 + h, h)
    trap = np.trapezoid(xs**2, xs)
    print(h, trap, abs(trap - 1/3))
```

例 3：ODE 欧拉 vs RK45。 $y' = -y$, $y(0) = 1$, 精确解 $y = e^{-t}$. 欧拉 $h = 0.1$ 在 $t = 1$ 得约 0.3487 (误差大); solve_ivp 用 RK45 得约 0.3679 (近真值)。

概率统计基础

本附录为第 4 章 Pandas 的描述统计、第 7 章 Statsmodels 的回归/时序与第 0 章成绩统计提供数学背景：分布、期望方差、推断 (检验/置信区间)、回归与时间序列，并补齐本科易混淆的难点 (中心极限定理、检验功效、多重比较)。

C.1 直觉故事：为什么不能只看平均分

两个班平均分都是 75：甲班 74~76 很集中，乙班一半 50 一半 100。平均分相同，但“不确定性”天差地别。所以描述数据至少要：**中心 + 离散 + 形状**。

- 均值 $\bar{x} = \frac{1}{n} \sum x_i$ ：常见但怕极端值；
- 中位数：排序中间值，稳健；
- 标准差 $s = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2}$ ：离散程度 (样本用 $n-1$)；
- 分位数/箱线图：看分布尾巴与离群点。

正文见： [0 前置基础-07 综合案例](#)、[4 Pandas -01 基础数据结构](#)。

C.2 随机变量与分布 (讲解)

C.2.1 期望与方差

- $E[X] = \sum x p(x)$ ：长期平均。掷骰子期望 3.5——它是“中心裁判”，不是某个具体结果；
- $\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$ ：偏离平方的平均；
- 线性性质： $E[aX + b] = aE[X] + b$ ；独立时 $\text{Var}(aX + b) = a^2 \text{Var}(X)$ 。

C.2.2 常用分布

分布	记号	直觉	场景
正态	$N(\mu, \sigma^2)$	对称钟形	测量误差、均值近似
二项	$B(n, p)$	n 次独立试验成功次数	点击率、产品合格数
泊松	$\text{Pois}(\lambda)$	单位时间事件数	订单到达、故障次数
均匀	$U(a, b)$	等概率	随机数、先验
t/F/ χ^2	$t(n), F, \chi^2$	小样本推断	检验、ANOVA

C.2.3 中心极限定理（必须理解）

大量**独立同分布**随机变量之和（或均值）的分布近似正态，与原始分布无关：

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \xrightarrow{n \rightarrow \infty} N(0, 1)$$

这解释了**为什么样本均值越稳**、为什么 t 检验/置信区间成立。注意它要求“独立同分布”：时间序列相邻样本相关时不满足，不能直接套。

C.2.4 协方差与相关

$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$ ；相关系数 $\rho = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1]$ 。 $\rho = 1$ 完全正线性相关；
 $\rho = 0$ 仅表示**非线性**相关（可能存在非线性关系！）。

C.3 统计推断（重点、难点）

C.3.1 置信区间：从“估计值”到“可靠性”

均值置信区间：

$$\bar{x} \pm t_{\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}$$

- $s/\sqrt{n}$ = 标准误：均值的“抖动范围”；
- 乘 t 分位数把“1 个标准误”扩成“95% 覆盖”；
- 样本越大/波动越小 → 区间越窄 → “越确信”。

C.3.2 假设检验：p 值到底在说什么

流程：立 H_0 → 选统计量 → 算 p 值（在 H_0 下看到这么极端数据的概率）→ 与 $\alpha = 0.05$ 比较。

检验	回答的问题
单/双样本 t	均值是否等于/不同
卡方	分类拟合、列联独立

检验	回答的问题
ANOVA (F)	多组均值是否全等
KS	分布是否一致

两类错误： α （第一类，误拒）、 β （第二类，漏拒）；**功效** $1 - \beta$ 。样本太小 $\rightarrow$ 功效低 $\rightarrow$ “不显著”可能只是没测出来，不代表“没差异”。

多重比较：一次做 20 次检验，纯运气也可能“显著”几次 $\rightarrow$ 用 Bonferroni/FDR 校正。

C.3.3 检验实操清单

1. 问题类型 $\rightarrow$ 选 t/卡方/ANOVA/KS/ADF;
2. 样本独立？配对数据用配对检验；
3. 正态性：小样本看 QQ 图/Shapiro；偏态用非参数或自助法；
4. 多重比较校正；
5. 报告： p 值 + 效应量 + 置信区间。

C.4 回归与方差分析（建模视角）

C.4.1 线性回归 = 换视角的最小二乘

模型 $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k + \varepsilon$ ；OLS 解：

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

要求（推断时）：线性、独立、同方差、残差正态。违背时用稳健标准误/变换。

C.4.2 怎么评价

- $R^2 = 1 - \text{SSE}/\text{SST}$ ：解释变异比例（过高要警惕过拟合）；
- 系数符号、大小、 p 值、置信区间：每个变量的“作用方向/强度/显著性”；
- **残差图**：比 R^2 更诚实（看趋势/异方差/离群点）。

手算：(1, 2), (2, 4), (3, 7) 一元回归斜率约 2.5、截距约 -0.5。

C.4.3 ANOVA：多组到底有没有差异

$$F = \frac{\text{组间方差}/(\text{组数}-1)}{\text{组内方差}/(\text{n}-\text{组数})}$$

F 大 = “组间差异比组内随机波动更明显”。

C.5 时间序列：和“普通回归”有什么不同（重点）

概念	直觉	工具
平稳性	均值/方差不随时间漂移	ADF 单位根检验
自相关	现在与过去的相关	ACF/PACF
差分	$y_t - y_{t-1}$	消除趋势
移动平均	局部平均去噪	rolling
分解	趋势 + 季节 + 残差	seasonal_decompose
ARIMA	AR + 差分 + MA	预测

关键：相邻样本相关 → 不能当独立样本做普通回归；先 ADF/差分，再看 ACF/PACF 定阶，否则 p 值“虚假显著”。

C.6 动手例题（选做）

例：置信区间。样本 [68, 91, 55, 76, 88]：均值 75.6，标准差约 14.0；95% 置信区间 $75.6 \pm t_{0.025,4} \times 14.0/\sqrt{5} \approx [58.2, 93.0]$ （很宽 → 样本太小）。代码：

```
import numpy as np
from scipy import stats
x = np.array([68, 91, 55, 76, 88])
print(x.mean(), x.std(ddof=1))
stats.t.interval(0.95, df=len(x)-1, loc=x.mean(), scale=stats.sem(x))
```

例：相关 ≠ 因果。 $y = x + \text{噪声}$ 相关系数接近 1；若有共同原因 z 同时影响二者，高相关只是巧合 → 建模要控制混杂。

C.7 Python 对应（速查）

```
import numpy as np, pandas as pd, statsmodels.api as sm
from scipy import stats

x = np.array([68, 91, 55, 76, 88])
pd.Series(x).describe()
stats.ttest_1samp(x, 60)
stats.chisquare([10, 20, 30])

X = sm.add_constant(np.array([[1.,2.],[2.,4.],[3.,7.])))
model = sm.OLS(np.array([2.,4.,7.]), X).fit()
model.summary()

from statsmodels.tsa.seasonal import seasonal_decompose
df = pd.read_csv("data.csv", index_col=0, parse_dates=True)
seasonal_decompose(df["sales"], model="additive").plot()
```


你想要	用哪个
描述统计	<code>Series.describe()</code>
分布检验	<code>scipy.stats</code>
回归/ANOVA	<code>statsmodels.OLS / ols</code>
时序分解	<code>seasonal_decompose</code>
ARIMA	<code>statsmodels.tsa.arima.model.ARIMA</code>

C.8 常见误区

误区	正确
p 值小 = 影响大	p 值与效应量是两回事
只看均值	先看分布（直方图/箱线图）
非平稳直接回归	先 ADF/差分
相关 = 因果	相关 $\neq$ 因果
小样本硬用正态近似	用小样本检验/重抽样
不显著 = 没有差异	可能功效不足（样本小）

C.9 使用章节（双向）

章	哪里用到	链接
0 前置基础	成绩统计	07 综合案例
4 Pandas	分组聚合/描述统计	01 基础数据结构
7 Statsmodels	回归/ANOVA/时序	01 方差分析与回归
3 SciPy	假设检验	04 假设检验工具包

下游衔接：intro-mathmodel 第 7 章（权重/评价）、第 8 章（时间序列）、第 9 章（机器学习统计模型）。**延伸阅读：**statsmodels 官方文档、ThinkStats2、《概率论与数理统计》（见 [references.md](#)）。

C.12 常见考题与自查（考前 10 分钟）

会了吗？	考点	一句话答案
☐	p<0.05 的含义	在 H0 下看到这么极端数据的概率小
☐	p 小 $\neq$ 影响大	看效应量与置信区间
☐	置信区间与样本量	n 越大区间越窄（标准误 $s/\sqrt{n}$ ）
☐	中心极限定理	大量独立同分布均值近似正态

会了吗？ 考点	一句话答案
☐ 相关 $\neq$ 因果	可能有混杂变量
☐ 非平稳直接回归	先 ADF/差分，否则 p 值虚假
☐ 功效低	“不显著”可能只是样本不足

C.11 综合案例：一份“完整统计报告”的骨架

问题：某班两组练习方式（A/B）的成绩是否不同？影响分数的因素还有哪些？

步骤：

1. **描述统计：**每组均值/中位数/标准差/箱线图（C.1）；
2. **差异性检验：**先检查正态性（Shapiro），再用独立样本 t 检验或 Mann-Whitney；报告 p 值与置信区间（C.3）；
3. **回归：**把“练习方式”与“前测成绩”放进线性回归，看系数与 p 值（C.4）；
4. **时序（若有）：**对周销量做分解/ADF，再决定用差分或 ARIMA（C.5）；
5. **结论模板：**“在控制了 X 后，A 组平均高 delta 分（95%CI [...]），p=...；但样本量小，需谨慎解释。”

参考要点代码：

```
import pandas as pd, statsmodels.api as sm
from scipy import stats
df = pd.read_csv("scores.csv")
a = df.loc[df.group == "A", "score"]; b = df.loc[df.group == "B", "score"]
print(stats.ttest_ind(a, b))
X = sm.add_constant(pd.get_dummies(df[["group", "pretest"]]))
print(sm.OLS(df.score, X).fit().summary())
```

反思：统计报告不是“跑一个显著性”就完事；分布、检验假设、效应量、置信区间都要交代。

C.10 例题集（深入练习）

例 1：t 检验完整流程（手算路径）。样本 [68, 91, 55, 76, 88]，检验 $H_0: \mu = 60$ 。均值 75.6，标准误 $s/\sqrt{n} \approx 14.0/2.236 \approx 6.26$ ， $t = (75.6 - 60)/6.26 \approx 2.49$ 。查 df = 4 的 t 表，双尾 $p \approx 0.067 > 0.05$ → 不显著（样本太小）。

例 2：ANOVA 思路（三组）。组均值差很大、组内波动很小 → F 大 → 拒绝“三组均值全等”。代码：

```
import pandas as pd, statsmodels.api as sm
from statsmodels.formula.api import ols
df = pd.DataFrame({"score": [70, 75, 72, 80, 85, 83, 60, 62, 58],
                  "group": ["A"]*3 + ["B"]*3 + ["C"]*3})
print(ols("score ~ C(group)", data=df).fit().summary()) # 看 F 与 p
```

例 3：ADF 读数。对非平稳序列做 ADF， $p > 0.05$ → 不能拒绝“存在单位根”（非平稳）→ 先差分再建模。

图论与网络

本附录为第 6 章 NetworkX 提供图论与算法背景：图表示、基本量、经典算法（BFS/最短路/MST/中心性/社区），并补上本科常见难点（握手定理、树的性质、模块度）。

D.1 直觉故事：朋友圈里的“谁最重要”

“谁最重要”没有唯一答案：认识 10 个互相不认识的人 → 桥梁；认识 10 个彼此都认识的人 → 不是桥梁。所以：

- **度中心性**：连接数多 → “活跃”；
- **接近中心性**：到所有人平均距离短 → “信息传得快”；
- **介数中心性**：很多最短路经过它 → “枢纽/瓶颈”；
- **PageRank**：被“重要的人”指向 → “影响力”。

做网络分析第一步是**明确要问什么**，再选指标。

正文见：[6 NetworkX ·02 分析图](#)、[6 NetworkX ·03 求解图的基本问题](#)。

D.2 图的基本表示（讲解 + 手算）

D.2.1 怎么把图“喂给电脑”

表示	样子	适合
邻接矩阵 A	$A[i][j] = 1$ （或权重）	稠密图、矩阵运算；点多费内存
邻接表	每个点存邻居列表	稀疏大图；遍历快

手算例子（4 点：A-B、A-C、B-C、C-D）：

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

（无向图对称；`np.linalg.eigvalsh(A)` 能直接读图的性质，如连通性/谱聚类。）

D.2.2 基本量：度、连通、树

- **度 $\deg(v)$** ：邻边数；有向图分**入度/出度**；
- **握手定理**： $\sum_v \deg(v) = 2|E|$ （每条边贡献 2 度）——检查数据是否“边数对着”；
- **路径/连通分量**：可达性；**树**：无环连通图， n 个点恰 $n - 1$ 条边；
- **二分图**：点分两类，边只在类间（学生-选课、用户-商品）。

D.2.3 加权与有向

- **加权图**：边带距离/成本/强度；最短路、MST、网络流依赖这些权；
- **有向图**：边有方向（关注、引用、资金流）；PageRank/拓扑排序只在有向图上。

D.3 经典算法：思路与手算

D.3.1 遍历：BFS 与 DFS

- **BFS**：一层层扩散。无权图最短路靠它；手算：从 A 出发，A→B、C，再 C→D，路径 A→C→D 两步最短；
- **DFS**：一条路走到黑再回头；用于检测环、连通分量、拓扑排序。

D.3.2 最短路径

算法	适用	思路	复杂度
BFS	无权	按层扩散	$O(V + E)$
Dijkstra	非负权	每次取当前最近点并松弛	$O((V + E) \log V)$ (堆)
Bellman-Ford	可能有负权	反复松弛	$O(VE)$
Floyd-Warshall	全源	动态规划	$O(V^3)$

手算 Dijkstra (A-B=2、A-C=5、B-C=1、C-D=4)：A=0 → B=2、C=5 → 取 B=2，松弛 C 得 3 → 取 C=3，松弛 D=7 → 得 A→B→C→D，距离 7。

D.3.3 最小生成树 (MST)

目标：总权重最小的连通子图（树）。**Kruskal**：边按权排序，用并查集“不成环就加”；最后正好 $n - 1$ 条边。**Prim**：从一个点出发，每次加入“连接已选集合的最便宜边”。两者都满足“贪心 + 无环”的树性质。

D.3.4 中心性与社区

- **介数中心性**：统计每条最短路经过某点的次数；
- **PageRank**：随机游走稳态；链接不仅“数量”还看“质量”；
- **社区发现**：把网络切成“内部紧密、外部稀疏”。**模块度**：

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{d_i d_j}{2m} \right] \delta(c_i, c_j)$$

直觉：实际边数 - 随机网络下的期望边数（同一社区内）。**Louvain** 贪心优化模块度，快且常用；结果与分辨率/随机种子有关，**要报告方法与参数**。

D.4 动手例题（选做）

例：Dijkstra 代码验证：

```
import networkx as nx
G = nx.Graph()
G.add_weighted_edges_from([("A","B",2), ("A","C",5), ("B","C",1), ("C","D",4)])
print(nx.shortest_path(G, "A", "D", weight="weight"))           # ['A', 'B', 'C', 'D']
print(nx.shortest_path_length(G, "A", "D", weight="weight"))    # 7
```

例：中心性完全不同。星型网络（中心 A 连 5 个叶子）：A 的介数中心性远高于叶子；但每个叶子的度都是 1——“度中心性”看不出谁是枢纽。

D.5 Python 对应（速查）

```
import networkx as nx
G = nx.Graph()
G.add_edges_from([("A","B",{ "w":2}), ("A","C",{ "w":5}),
                  ("B","C",{ "w":1}), ("C","D",{ "w":4})])

nx.shortest_path(G, "A", "D", weight="w")           # Dijkstra
nx.shortest_path_length(G, "A", "D", weight="w")
nx.minimum_spanning_tree(G, weight="w")
nx.degree_centrality(G); nx.betweenness_centrality(G)
nx.pagerank(G)
nx.community.louvain_communities(G)
nx.draw(G, with_labels=True)
```

你想要	用哪个
最短路	nx.shortest_path (加权传 weight)
MST	nx.minimum_spanning_tree
中心性	degree / closeness / betweenness / pagerank
社区	nx.community
可视化	nx.draw (或结合 Matplotlib)

D.6 常见误区

误区	正确
以为图只能是邻接矩阵	稀疏大图用 NetworkX 的邻接表实现
无向图当成有向图算	Graph 与 DiGraph 分开
用最短路回答“谁最重要”	先明确“重要”的定义
忽略权重	加权算法要传 weight
社区数量随便定	社区发现与分辨率有关；试多组参数并看模块度
握手定理没验证	先看 sum deg(v) == 2* E 排查数据

D.7 使用章节（双向）

章	哪里用到	链接
6 NetworkX	建图/分析/最短路	01 创建图
6 NetworkX	中心性/社区	02 分析图
6 NetworkX	综合案例	04 综合案例

下游衔接：intro-mathmodel 第 4 章（复杂网络与图论模型）。**延伸阅读：**NetworkX 官方教程、图论教材、[references.md](#)。

D.10 常见考题与自查（考前 10 分钟）

会了吗？	考点	一句话答案
☐	BFS 与 DFS 适用	BFS 无权最短路；DFS 环/连通/拓扑
☐	Dijkstra 前提	边权非负
☐	MST 贪心	Kruskal/Prim 每次选不形成环的最便宜边
☐	度 vs 介数中心性	度 = 活跃；介数 = 桥梁/瓶颈
☐	模块度	社区内实际边 - 随机期望，越大越“像社区”
☐	握手定理	度之和 = 2× 边数

D.9 综合案例：给一个小网络做“体检”

问题：给定学校 20 人社交网络，找出“信息传播最快的人”、“最容易被绕过的人”与“天然小团伙”。

步骤：

1. 建图：读边表，检查握手定理（度之和 = 2× 边数）；
2. 最短路：无权图用 BFS，加权用 Dijkstra；
3. 中心性：同时看度/接近/介数/PageRank，画四张图对比（结论不同是正常的，要把定义说清）；
4. 社区：Louvain 跑几组分辨率，报告模块度与参数；
5. 输出：“若做疫情通报，优先通知介数中心性最高的人；若做班委选举，关注度中心性与 PageRank 都高的人。”

参考要点代码：

```
import networkx as nx
G = nx.read_edgelist("edges.txt")
print(sum(d for _, d in G.degree()) == 2 * G.number_of_edges())
print(nx.betweenness_centrality(G)); print(nx.pagerank(G))
from networkx.algorithms.community import louvain_communities
print(louvain_communities(G, seed=0))
```

反思：图分析“结论”依赖指标定义；把“问什么”和“用什么”一起写进报告。

D.8 例题集（深入练习）

- 例 1：Kruskal 手算。**边 (A-B=2, A-C=5, B-C=1, C-D=4)：排序 B-C(1) → A-B(2) → C-D(4) → A-C(5)。依次加入不成环的边：B-C、A-B、C-D；此时 4 点 3 边成树，总权 7。A-C 会成环，跳过。
- 例 2：PageRank 三点手算**（有向环 A→B→C→A 加自环简化不计）：稳态分布与转移矩阵最大特征向量相同——直观上“每个点的权重等于流入它的点的权重之和”。
- 例 3：BFS 手算。**图 A-B、A-C、B-D、C-D：BFS 从 A 出发的层序：0 层 A；1 层 B、C；2 层 D。因此 A 到 D 最短路为 2（经 B 或 C），无权图最短路 = 层数。

机器学习基础

本附录为第 8 章 scikit-learn 提供数学框架：学习范式、损失与优化、评估与过拟合、模型直觉；并接上附录 A（SVD/PCA）与附录 B（优化）。补上本科难点：偏差-方差分解、交叉熵直觉、正则化几何。

E.1 直觉故事：为什么模型会“学偏”

用“学习时长”预测“考试成绩”，数据近似一条直线，线性回归找“斜率 + 截距”。

但只有 5 个学生、加了 10 个特征（连“头发长度”都算），线性回归能**完美穿过所有 5 个点**——换一批学生就崩。这就是**过拟合**：记住训练集，没学普适规律。

核心矛盾：**训练集上够准 + 新数据上也准**。靠“任务框架（损失）+ 优化 + 验证评估 + 正则化/适当复杂度”解决。

正文见：[8 sklearn ·01 数据集的预处理](#)、[8 sklearn ·02 有监督学习的案例](#)。

E.2 学习范式与损失函数（讲解）

E.2.1 三大任务

任务	输入/输出	常见模型
回归	(x, y) , y 连续	线性回归、Ridge/Lasso、决策树、SVR
分类	(x, y) , y 离散	逻辑回归、决策树、随机森林、SVM
聚类	只有 x	k-Means、层次聚类、DBSCAN
降维	只有 x	PCA (SVD)、t-SNE、UMAP

统一视角：**模型 = 假设集 + 损失函数 + 优化算法**。

E.2.2 损失函数：怎么算“错得有多离谱”

- **回归•MSE**: $L = \frac{1}{n} \sum (y_i - f(x_i))^2$ ——对离群点敏感；RMSE 是平方根，量纲与 y 一致；
- **回归•MAE**: $\frac{1}{n} \sum |y_i - f(x_i)|$ ——更稳健；
- **分类•交叉熵**: $L = - \sum y_i \log p_i$ ——鼓励“预测概率对且自信”；与逻辑回归天然搭配（sigmoid 的梯度形式漂亮）；
- **聚类•k-Means**: 点到最近中心的平方距离之和（WCSS），肘部法选 k 。

E.2.3 优化：把损失调小

梯度下降： $\theta \leftarrow \theta - \alpha \nabla L(\theta)$ 。

手算： $L(\theta) = (\theta - 3)^2$, $\theta_0 = 1$, $\nabla L = 2(\theta - 3) = -4$ 。 $\alpha = 0.5 \rightarrow$ 一步到 3; $\alpha = 2 \rightarrow$ 弹到 9——学习率太大震荡。实际还要做特征缩放（不同特征“感受”不同学习率）。

正则化（几何视角）：Ridge 加 $\lambda \|\theta\|^2$ （系数不过大）；Lasso 加 $\lambda \|\theta\|_1$ （鼓励稀疏）。本质是在“拟合误差”与“解的大小”之间取舍； λ 越大越“保守”。

E.3 评估与过拟合（重点、难点）

E.3.1 训练/验证/测试

- 训练集：学参数；验证集/交叉验证：挑超参数；测试集：最后评估一次；
- 交叉验证（ k -fold）：数据切 k 份轮流验证，结果更稳；
- **别用测试集反复调参**——等于把测试集“教”给模型。

E.3.2 偏差-方差分解（必考理解）

- $$E[(f - \hat{f})^2] = \underbrace{\text{Bias}^2}_{\text{欠拟合}} + \underbrace{\text{Var}}_{\text{过拟合}} + \sigma^2$$
- 太简单 $\rightarrow$ 高偏差（欠拟合）；太复杂 $\rightarrow$ 高方差（过拟合）；噪声 σ^2 不可约；
 - 调复杂度（深度/正则/特征数）就是找偏差-方差平衡点。

E.3.3 指标怎么选

任务	指标	注意
回归	MSE/RMSE/MAE/ R^2	结合业务解读
分类	准确率/精确率/召回率/F1/ROC-AUC	类别不平衡别只看准确率
聚类	轮廓系数/CH/DB	与 k 、距离度量有关
降维	解释方差比/重构误差	先标准化再 PCA

E.4 常用模型直觉（讲解 + 速查）

模型	一句话直觉	关键超参
线性回归	找一条“最不怨”的直线	无；特征缩放
Ridge/Lasso	给斜率“限高”	α
逻辑回归	线性分数 $\rightarrow$ sigmoid $\rightarrow$ 概率	C （正则强度）
决策树	按特征反复二分	max_depth 、 min_samples_leaf
随机森林	多棵随机树投票	$n_{\text{estimators}}$ 、 max_depth
SVM	最大化分类间隔	kernel、 C
kNN	看最近的 k 个邻居	k 、特征缩放（必须）

模型	一句话直觉	关键超参
k-Means	迭代挪中心	k 、 n_{init} 、种子
PCA	找方差最大的正交方向	$n_{\text{components}}$ 、先标准化

PCA 的数学就是附录 A 的 SVD：特征矩阵做 SVD，右奇异向量即主成分方向；取前 k 个 = 最优低秩近似。

E.5 动手例题（选做）

例：过拟合实验。8 个点， $\text{degree} = 1$ （直线）与 $\text{degree} = 9$ （9 次多项式）：

- 直线：训练 RMSE 大一点，但换数据也稳；
- 9 次：训练 RMSE 几乎为 0，但预测新点剧烈摆动——“训练好 $\neq$ 泛化好”。

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
X = np.sort(rng.uniform(0, 1, 8))[:, None]
y = np.sin(6 * X[:, 0]) + rng.normal(0, 0.1, 8)
for deg in [1, 9]:
    m = make_pipeline(PolynomialFeatures(deg), LinearRegression()).fit(X, y)
    print(deg, mean_squared_error(y, m.predict(X)) ** 0.5)
```

例：为什么先做特征缩放。“体重 (kg) 50~100”+ “身高 (m) 1.5~2.0”：距离/梯度被体重主导。用 StandardScaler 后两者同量纲，kNN/PCA/梯度类模型才可靠。

E.6 Python 对应（scikit-learn 标准流程，速查）

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, f1_score, roc_auc_score, silhouette_score)
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

X, y = ...
X = StandardScaler().fit_transform(X)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=0)

clf = LogisticRegression(C=1.0, max_iter=1000).fit(Xtr, ytr)
print(accuracy_score(yte, clf.predict(Xte)),
      f1_score(yte, clf.predict(Xte), zero_division=0),
      roc_auc_score(yte, clf.predict_proba(Xte)[:, 1]))
```

```
cross_val_score(LogisticRegression(), X, y, cv=5)
KMeans(n_clusters=3, n_init=10, random_state=0).fit(X)
PCA(n_components=2).fit_transform(X)
```

你想要	用哪个
划分/缩放	train_test_split / StandardScaler
评估	cross_val_score + 指标函数
聚类	KMeans + silhouette_score
降维	PCA
特征重要性	树模型 feature_importances_

E.7 常见误区

误区	正确
不看数据直接建模	先 EDA、缺失值、异常值、缩放
反复用测试集调参	用验证/交叉验证，测试集只碰一次
类别不平衡只看准确率	看 F1/AUC/PR
PCA 前不标准化	方差大的特征主导；先 StandardScaler
聚类只跑一遍	结果与初始化有关；固定种子、多 n_init
模型越复杂越好	交叉验证选“够用”的复杂度

E.8 使用章节（双向）

章	哪里用到	链接
8 sklearn	预处理/监督/无监督	01 数据集的预处理
8 sklearn	模型训练与评估	02 有监督学习的案例
8 sklearn	聚类/PCA	03 无监督学习的案例
5 Matplotlib	评估可视化	01 基本绘图

数学底子：附录 A (SVD/PCA)、附录 B (优化)、附录 C (统计推断/回归)。**下游衔接：**intro-mathmodel 第 9 章 (机器学习与统计模型)。**延伸阅读：**scikit-learn User Guide、mml-book Part II、《机器学习》周志华 (见 [references.md](#))。

E.11 常见考题与自查（考前 10 分钟）

会了吗？	考点	一句话答案
□	过拟合的表现	训练好、测试差；高方差
□	欠拟合	训练就不好；高偏差
□	偏差-方差分解	总误差 = 偏差 ^2 + 方差 + 噪声
□	类别不平衡看什么	F1 / AUC，不看准确率
□	正则化作用	限制系数大小，防过拟合
□	k 怎么选	肘部法 + 轮廓系数；固定种子多试
□	PCA 前为什么缩放	大数值特征主导方差

E.10 综合案例：一次完整的建模流程（期末大作业可直接套）

问题：用订单数据预测“销售额”，并解释模型。

步骤：

- 1. **EDA**：缺失值、离群点、分布（直方图/箱线图）；
- 2. **特征**：类别编码、数值缩放；训练/测试 8:2，固定种子；
- 3. **基线**：线性回归先跑，看 RMSE 与 R^2；
- 4. **交叉验证**：5 折，比较线性/岭/树模型，选“够用”的；
- 5. **调复杂度**：对树模型调 max_depth，对岭模型调 alpha（用验证集，不动测试集）；
- 6. **解释**：特征重要性/系数符号、残差图，写 3 条业务结论；
- 7. **最终**：测试集只跑一次，报告指标与局限性。

参考要点代码：

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
Xs = StandardScaler().fit_transform(X)
Xtr, Xte, ytr, yte = train_test_split(Xs, y, test_size=0.2, random_state=0)
m = Ridge(alpha=1.0).fit(Xtr, ytr)
scores = cross_val_score(Ridge(alpha=1.0), Xtr, ytr, cv=5,
                          scoring="neg_mean_squared_error")

print(scores) print(mean_squared_error(yte, m.predict(Xte)) ** 0.5)
```

反思：把附录 E 的“损失/评估/过拟合”变成一套可复用流程，期末大作业与科研都适用。

E.9 例题集（深入练习）

例 1：梯度下降手算 3 步。 $L(\theta) = (\theta - 3)^2$, $\alpha = 0.5$: $\theta_0 = 1 \rightarrow \theta_1 = 3$ （一步到最优）。若 $\alpha = 0.9$: $1 \rightarrow 4.6 \rightarrow 2.48 \rightarrow 3.42 \rightarrow 2.82 \rightarrow 3.07 \rightarrow \dots$ ——震荡收敛。结论：步长太大振荡、太小慢。

例 2：逻辑回归 sigmoid 手算。 $\theta^T x = 0 \rightarrow p = 0.5$ （边界）； $= 2 \rightarrow p \approx 0.88$ ； $= -2 \rightarrow p \approx 0.12$ ——线性分数越大，概率越接近 1。

例 3: R^2 手算。数据 $(0, 1), (1, 2), (2, 3)$, 拟合 $\hat{y} = x + 1$ 无残差: $SSE=0$, $SST=2$ (y 的方差 $\times n$), $R^2 = 1$ 。若 $\hat{y} = 2$ (常数预测): $SSE=2$, $SST=2$, $R^2 = 0$ ——“不比平均更好”。

```
from sklearn.metrics import r2_score
print(r2_score([1,2,3], [1,2,3]))    # 1.0
print(r2_score([1,2,3], [2,2,2]))    # 0.0
```

例 4: 交叉验证的意义。只用一次 hold-out 可能“切巧了”; 5 折交叉验证给出 5 个分数的平均与波动, 更值得信。

通用数学与算法速查

本附录是“随手翻”的卡片: 常用记号、公式、复杂度、数值稳定性、可视化原则与术语英中对照。建议上机前翻 5 分钟, 期末大作业前完整过一遍。

F.1 常用记号与希腊字母 (速查)

记号	含义	记号	含义
$\sum$	求和	$\prod$	连乘
$E[X]$	期望	$\text{Var}(X)$	方差
x^T	转置	∇f	梯度
$\partial f / \partial x$	偏导	$\int$	积分
λ	特征值/正则强度	α	学习率
θ	参数/角度	ε	误差/小量
$\approx$	约等于	$\ll$	远小于

读公式的口号: 先找求和/连乘范围, 再看指数/对数在谁身上, 最后看变量是矩阵还是标量。

F.2 常用公式卡 (速查)

微积分

- $(x^n)' = nx^{n-1}; (e^x)' = e^x; (\ln x)' = 1/x$
- 泰勒: $f(x+h) \approx f(x) + f'(x)h + \frac{1}{2}f''(x)h^2$
- 数值微分 (中心差分): $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$

线性代数

- $(AB)^T = B^T A^T; \text{tr}(AB) = \text{tr}(BA)$
- 最小二乘: $x^* = \arg \min_x \|Ax - b\|_2^2$ (数值用 `lstsq`)
- SVD: $A = U\Sigma V^T$; 对称矩阵: $A = Q\Lambda Q^T$

概率统计

- $E[X] = \sum x p(x)$; $\text{Var}(X) = E[X^2] - (E[X])^2$
- 样本方差: $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$
- 置信区间: $\bar{x} \pm t_{\alpha/2, n-1} \cdot s / \sqrt{n}$
- 线性回归: $y = X\beta + \varepsilon$, $\hat{\beta} = (X^T X)^{-1} X^T y$, $R^2 = 1 - \text{SSE}/\text{SST}$

机器学习

- 逻辑回归 (sigmoid): $p = 1/(1 + e^{-\theta^T x})$; 损失 = 交叉熵
- 梯度下降: $\theta \leftarrow \theta - \alpha \nabla L$
- 正则化: Ridge $L + \lambda \|\theta\|^2$; Lasso $L + \lambda \|\theta\|_1$

F.3 复杂度与常用算法量级（速查）

运算	复杂度	备注
矩阵乘/解方程组 ($n \times n$)	$O(n^3)$	向量化库已优化; 避免显式求逆
特征分解 / SVD	约 $O(n^3)$	只求前 k 个可加速
FFT	$O(N \log N)$	比 $O(N^2)$ 直接 DFT 快得多
Dijkstra	$O((V + E) \log V)$	堆实现
Kruskal MST	$O(E \log E)$	排序为主
k-Means 一次迭代	$O(n \cdot k \cdot d)$	与样本数线性
Python 纯循环	慢	用 NumPy 向量化/广播

实践建议: 先写对, 再量 (%timeit); 大数据用向量化, 循环只在必要时。

F.4 数值稳定性（科学计算必修，讲解）

浮点数是有限精度（约 15~16 位有效小数），“数学上对的公式”在电脑上可能“崩”：

1. **比较:** 别用 `==` 比浮点; 用 `np.isclose` / $|a - b| < 10^{-9}$;
2. **条件数:** $\text{cond}(A)$ 大 $\rightarrow$ 病态, 解对输入过敏;
3. **灾难性消去:** $\sqrt{x^2 + 1} - x$ (大 x) $\rightarrow$ 改写 $1/(\sqrt{x^2 + 1} + x)$;
4. **缩放:** 优化/距离/聚类前标准化 (StandardScaler);
5. **对数域:** 连乘小概率取 `log` 防下溢; 分类用 `log_proba`;
6. **可复现:** 固定种子 (`random_state` / `default_rng(seed)`)。

手算直觉: $x = 10^8$ 时 $\sqrt{x^2 + 1} - x$ 双精度算出来约 0 (真值约 5×10^{-9}) ——不是数学错了, 是浮点“看不见”那么小的差。

F.5 关键词索引（速查）

你想查	去哪
矩阵/分解/最小二乘	附录 A
导数/积分/优化/插值/FFT/ODE	附录 B
分布/检验/回归/时序	附录 C
图/路径/中心性/社区	附录 D
损失/评估/模型/过拟合	附录 E
记号/公式/复杂度/稳定性	本附录 F
全书目	附录 G · 全书总参考文献

F.6 使用章节（双向）

章	用法	链接
0 前置基础	成绩统计公式	07 综合案例
1 NumPy	数值稳定性/向量化	02 数组运算
3 SciPy	优化/插值上下界	02 优化工具包
5 Matplotlib	可视化原则	01 基本绘图
8 sklearn	特征缩放/评估	01 数据集的预处理

可视化原则（配合第 5 章）：一图一信息；坐标轴标清楚；柱状看类别、直方看分布、散点看关系、折线看趋势；标注单位/图例/样本量；颜色兼顾色盲。

F.7 术语英中对照（速查）

英文	中文	出现在
matrix / vector	矩阵 / 向量	附录 A
eigenvalue / singular value	特征值 / 奇异值	附录 A、E
least squares	最小二乘	附录 A、C、E
gradient / Hessian	梯度 / 黑塞矩阵	附录 B、E
interpolation / fitting	插值 / 拟合	附录 B
hypothesis test / p-value	假设检验 / p 值	附录 C
confidence interval	置信区间	附录 C
central limit theorem	中心极限定理	附录 C
shortest path / MST	最短路 / 最小生成树	附录 D
centrality / community	中心性 / 社区	附录 D
loss / regularization	损失 / 正则化	附录 E
overfitting / cross-validation	过拟合 / 交叉验证	附录 E
complexity / numerical stability	复杂度 / 数值稳定性	附录 F

一句话总结：遇到不认识的名词，先查这张表，再进对应主题读“直觉故事”。

F.9 考前 10 分钟清单（全附录）

- 1. 记号： $\sum$ 、 $E[X]$ 、 ∇ 、 A^T ——见 F.1;
- 2. 公式：导数/积分/最小二乘/置信区间/正则化——见 F.2;
- 3. 复杂度：矩阵乘 $O(n^3)$ 、FFT $O(N \log N)$ 、Dijkstra $O((V + E) \log V)$ ——见 F.3;
- 4. 数值稳定性：条件数、灾难消去、缩放、对数域——见 F.4;
- 5. 关键词索引：哪类问题看哪个附录——见 F.5;
- 6. 术语英中：matrix、loss、p-value.....——见 F.7;
- 7. 常用技巧：向量化、log 域、日志坐标、NaN 处理、种子——见 F.8。

最后一句：先看“直觉故事”建立画面，再用“速查表”拿公式，最后用“例题集”练手。

F.8 数值小技巧（速查）

- 1. 求和/连乘尽量用向量化；避免 Python 循环里重复分配大数组；
- 2. 概率乘积取对数：np.log + np.sum;
- 3. 画图前把大数值日志化 (plt.yscale("log"));
- 4. 长序列先 np.fft.fft 看谱，再决定滤波/平滑；
- 5. 拟合前先 np.nan_to_num/去掉缺失，避免 NaN 传染；
- 6. 复现：所有随机过程给 random_state/种子，并写进报告。

全书总参考文献（附录 G）

本页由 build/gen_references.py 自动生成，勿手改。单一事实源 = 各章 references.md + 附录 references.md；新增/删除引用后重新运行生成即可。速查方式：按类别浏览；每行“用于”列标注了哪些章节使用该资料，点章节链接可回正文。

共汇总 195 条（来自 9 个章节 + 附录）。

一、官方文档与工具链

资料	说明	用于
API Reference	全部函数索引	第 7 章 Statsmodels
ARIMA	ARIMA/SARIMAX 模型	第 7 章 Statsmodels
Anaconda vs Miniconda	选型说明（课程推荐 Miniconda）	第 0 章 前置基础
Broadcasting 官方说明	广播规则权威定义	第 1 章 NumPy
Complex Network Analysis in Python（官方示例）	官方示例集，覆盖绘图/社区/路径	第 6 章 NetworkX
Examples 总目录	官方示例（含回归与时间序列）	第 7 章 Statsmodels
Formula API	ols('y ~ x' , data=...) 语法	第 7 章 Statsmodels
GLM	广义线性模型	第 7 章 Statsmodels
Git 官方文档	书/命令/教程	第 0 章 前置基础
I/O 参考	save/loadtxt/savez	第 1 章 NumPy

资料	说明	用于
Jupyter 官方文档	笔记本/内核/导出	第 0 章 前置基础
Learn LaTeX 中文站	免费结构化课程：结构/公式/表格/插图	第 0 章 前置基础
LoongTeX（龙文）	国产在线编译（备选，中文友好）	第 0 章 前置基础
Matplotlib Cheatsheets	速查表（含多子图排版）	第 5 章 Matplotlib
Matplotlib Gallery 官方示例	每个图型含完整可运行源码	第 5 章 Matplotlib
Matplotlib 图型画廊	各类图型官方示例	第 5 章 Matplotlib
Matplotlib 快速开始	官方绝对入门，覆盖 Plot/Axes/Figure	第 5 章 Matplotlib
Matplotlib 文本/字体	字体、字号、数学公式	第 5 章 Matplotlib
Matplotlib: Anatomy of a Figure	子图排版与 tight_layout 细节	第 5 章 Matplotlib
NetworkX 3.x 迁移/新特性	版本更新说明	第 6 章 NetworkX
NetworkX Tutorial（官方教程）	从零创建图、访问节点边、常用操作	第 6 章 NetworkX
NetworkX 中文文档镜像	中文版官方文档（有翻译）	第 6 章 NetworkX
NetworkX 官方教程	API 与算法	附录
NetworkX 社区（Discussions/Stack Overflow）	常见问题与讨论入口	第 6 章 NetworkX
NumPy Quickstart	快速上手教程	第 1 章 NumPy
NumPy for MATLAB users	给 MATLAB 背景学生迁移	第 1 章 NumPy
NumPy 性能（broadcasting 等）	向量化与性能	附录
NumPy: Absolute Beginner	官方“绝对入门”，覆盖创建/索引/广播	第 1 章 NumPy
OLS	最小二乘回归	第 7 章 Statsmodels
Overleaf	在线编译（备选）	第 0 章 前置基础
Pandas 10 分钟入门	一分钟建立 Series/DataFrame 心智	第 4 章 Pandas
Pandas Cheat Sheet（官方）	一页速查	第 4 章 Pandas
Pandas 中文入门（pandas.pydata.org 官方中文翻译入口）	官方文档，含中文化镜像	第 4 章 Pandas
Pandas 官方主页	库主页、版本、安装	第 4 章 Pandas
Pandas 用户指南	索引/清洗/分组/时间序列全指南	第 4 章 Pandas
Pro Git 中文版	系统化学习	第 0 章 前置基础
Pyplot 教程	函数式接口与状态机说明	第 5 章 Matplotlib
Python 官方教程（中文）	语法查漏的第一权威	第 0 章 前置基础
Regressions	线性回归/GLS/WLS 总览	第 7 章 Statsmodels
SciPy 中文文档镜像	中文版参考（版本可能较旧）	第 3 章 SciPy
SciPy 主页	库介绍、版本、安装	第 3 章 SciPy
SciPy 参考手册（全模块）	按模块查函数签名与示例	第 3 章 SciPy
SciPy 官方教程（tutorial 目录）	官方讲解积分/FFT/插值	第 3 章 SciPy
SciPy 官方示例库	函数页内附示例	第 3 章 SciPy
SciPy 版本发布说明	查看函数改名/废弃	第 3 章 SciPy
Seaborn 官方教程	分布/箱线/热力图/分面教程	第 5 章 Matplotlib
Statsmodels in Python（教育向）	官方逐步教学	第 7 章 Statsmodels
Statsmodels 中文文档（ApacheCN 翻译）	官方文档的中文翻译	第 7 章 Statsmodels
Statsmodels 主页	库简介与总入口	第 7 章 Statsmodels
Statsmodels 官方示例 Notebooks	直接可跑的 notebook，含回归/诊断/时间序列	第 7 章 Statsmodels

资料	说明	用于
Statsmodels 官方示例（回归）	一整套可运行回归案例	第 7 章 Statsmodels
Statsmodels 官方示例（时间序列）	分解、ARIMA/SARIMA 案例	第 7 章 Statsmodels
SymPy Calculus（求导/积分/极限）	极限、diff、integrate 的权威说明	第 2 章 SymPy
SymPy Gotchas（常见坑）	官方 FAQ，避开符号计算陷阱	第 2 章 SymPy
SymPy Matrices	符号矩阵、特征值、分解	第 2 章 SymPy
SymPy ODE (dsolve)	微分方程符号求解	第 2 章 SymPy
SymPy Solvers	solve/solveset/linsolve/nonlinsolve 参考	第 2 章 SymPy
SymPy Tutorials（总目录）	官方入门教程入口，含基础与进阶	第 2 章 SymPy
SymPy lambdify	符号 → 数值函数的关键工具	第 2 章 SymPy
SymPy 官方 1.13 文档	完整 API 参考	第 2 章 SymPy
SymPy 官方 doc 中的 Examples	每条均可上机复现	第 2 章 SymPy
TSA（时间序列）	时间序列模块总览	第 7 章 Statsmodels
TeX Live 下载	Windows 安装器	第 0 章 前置基础
User Guide	全书式教学指南	第 7 章 Statsmodels
VS Code Python 入门	从新建文件到运行调试	第 0 章 前置基础
VS Code Python 环境官方文档	解释器/虚拟环境/内核原理	第 0 章 前置基础
VS Code 官方下载	编辑器	第 0 章 前置基础
WLS	加权最小二乘	第 7 章 Statsmodels
adfuller	ADF 单位根检验	第 7 章 Statsmodels
anova_lm	方差分析	第 7 章 Statsmodels
ctex 宏包文档	中文排版权威	第 0 章 前置基础
ndarray 基础	形状、索引、切片、视图/拷贝	第 1 章 NumPy
numpy.linalg 参考	线性代数全部函数	第 1 章 NumPy
numpy.linalg 官方参考	函数速查	附录
numpy.polynomial 参考	多项式接口	第 1 章 NumPy
pandas.DataFrame 参考	DataFrame 全部方法	第 4 章 Pandas
pandas.Series 参考	Series 全部方法	第 4 章 Pandas
pivot_table 参考	透视表	第 4 章 Pandas
scikit-learn API 速查	全部类/函数索引	第 8 章 scikit-learn
scikit-learn MOOC（INRIA）	官方协作的免费课程，带习题与 Notebook	第 8 章 scikit-learn
scikit-learn User Guide	官方模型/评估	附录
scikit-learn 交叉验证	train/test、交叉验证、调参	第 8 章 scikit-learn
scikit-learn 入门教程	官方快速上手，含“选择正确评估器”参考	第 8 章 scikit-learn
scikit-learn 官方主页 / 稳定版文档	权威 API 与用户指南入口，建议作为查错第一站	第 8 章 scikit-learn
scikit-learn 官方示例库	大量可运行示例，按算法/任务索引	第 8 章 scikit-learn
scikit-learn 模型评估	分类/回归/聚类指标	第 8 章 scikit-learn
scikit-learn 特征选择	过滤/包装/嵌入三类方法	第 8 章 scikit-learn
scikit-learn 用户指南	按主题讲解预处理、监督、无监督、模型选择	第 8 章 scikit-learn
scikit-learn 预处理	编码、缩放、缺失值处理	第 8 章 scikit-learn

资料	说明	用于
scipy.fft 傅里叶变换	fft/rfft/fftfreq	第 3 章 SciPy
scipy.integrate 积分与 ODE	quad/odeint/solve_ivp/solve_bvp	第 3 章 SciPy
scipy.interpolate 插值	一维/多维/散点插值	第 3 章 SciPy
scipy.optimize / scipy.integrate 官方文档	函数 API	附录
scipy.optimize 优化	求根/极值/规划/拟合	第 3 章 SciPy
scipy.signal 信号处理	滤波器设计、卷积	第 3 章 SciPy
scipy.stats 统计	检验/分布/描述统计	第 3 章 SciPy
seasonal_decompose	时间序列分解	第 7 章 Statsmodels
statsmodels 官方	方差分析事后比较/回归	第 3 章 SciPy
statsmodels 官方文档	回归/ANOVA/时序	附录
中心性算法	介数/接近/度中心性	第 6 章 NetworkX
分组 (groupby)	分组聚合/变换/过滤	第 4 章 Pandas
图与 NumPy/SciPy 互转	to_numpy_array / to_scipy_sparse_array	第 6 章 NetworkX
图生成器	path/cycle/complete/karate/随机图	第 6 章 NetworkX
图类型 (Graph / DiGraph 等)	Graph / DiGraph / MultiGraph 接口	第 6 章 NetworkX
安装 Miniconda	官方下载入口（速度慢用清华镜像）	第 0 章 前置基础
官方示例：空手道俱乐部	经典社区网络可视化	第 6 章 NetworkX
性能提升	向量化、chunksz 等	第 4 章 Pandas
时间序列 STL 示例	LOESS 季节性-趋势分解	第 7 章 Statsmodels
时间序列指南	date_range/resample/rolling	第 4 章 Pandas
最小生成树	minimum_spanning_tree 等	第 6 章 NetworkX
最短路径算法	Dijkstra / Bellman-Ford / Floyd-Warshall	第 6 章 NetworkX
社区检测与模块度	greedy_modularity / louvain / modularity	第 6 章 NetworkX
算法总览	全部算法模块入口	第 6 章 NetworkX
绘图 API	draw / spring_layout / 自定义	第 6 章 NetworkX
缺失值处理	isna/fillna/dropna/ffill/bfill	第 4 章 Pandas
连通性	is_connected / 强连通 / 分量	第 6 章 NetworkX
随机数 (Generator)	default_rng 用法	第 1 章 NumPy
面向对象接口	Figure/Axes 对象模型权威定义	第 5 章 Matplotlib

二、教材与书籍

资料	说明	用于
3Blue1Brown 线性代数的本质（中文字幕）	可视化直觉	附录
Mathematics for Machine Learning	Part I 数学基础（线性代数/微积分） + Part II 机器学习，主题式	附录
Python Data Science Handbook	数据科学视角，可作延伸	第 6 章 NetworkX

资料	说明	用于
Python Data Science Handbook	Jake VanderPlas 的《线性回归》一章对比 sklearn/statsmodels	第 7 章 Statsmodels
Python Data Science Handbook (Jake VanderPlas)	第 2 章 NumPy, 可整章对照	第 1 章 NumPy、第 3 章 SciPy、第 4 章 Pandas
Python Data Science Handbook (Jake VanderPlas)	第 4 章 Matplotlib、第 5 章 Seaborn, Notebook 形式	第 5 章 Matplotlib、第 8 章 scikit-learn
Python Data Science Handbook (符号计算附录)	第 5 章 SymPy 简介, 适合数据方向学生	第 2 章 SymPy
mml-book Chapter 2/3/4	线性代数/解析几何/矩阵分解	附录
thinkstats2	用 Python 讲统计	附录
课程仓库参考 (原: 南瓜书 (《机器学习》公式推导))	与周志华《机器学习》配套的公式推导详解	第 8 章 scikit-learn
课程仓库参考 (原: 开源社区 图深度学习 (葡萄书))	中文开源书, 进阶图神经网络	第 6 章 NetworkX
课程仓库参考 (原: 李宏毅机器学习 (开源社区整理))	中文讲义, 串起 ML 与 DL 的“为什么”	第 8 章 scikit-learn
课程仓库参考 (原: 李宏毅机器学习 (开源社区))	中文讲义, 机器学习到深度学习衔接	第 8 章 scikit-learn
课程仓库参考 (数学建模导论)	后续课程	第 1 章 NumPy、第 3 章 SciPy、第 4 章 Pandas
课程仓库参考 (数学建模导论: 第 1 章 Numpy 与线性代数)	下游对齐	附录
课程仓库参考 (数学建模导论: 第 4 章 复杂网络与图论模型)	下游对齐	附录
课程仓库参考 (数学建模导论: 第 7 章 权重/评价、第 9 章统计模型)	下游对齐	附录
课程仓库参考 (本课程主仓库)	本书开源仓库	第 1 章 NumPy、第 2 章 SymPy、第 3 章 SciPy、第 4 章 Pandas、第 5 章 Matplotlib、第 7 章 Statsmodels、第 8 章 scikit-learn

三、精品教程与课程

资料	说明	用于
100 NumPy Exercises (rougier)	100 题带解答, 适合课后	第 1 章 NumPy
Hands-On ML 3rd (Aurélien Géron)	第 2 章起系统讲 sklearn, 示例丰富	第 8 章 scikit-learn
Kaggle Intro to ML	用真实数据练决策树/随机森林/交叉验证	第 8 章 scikit-learn
Learn X in Y minutes: Python	半小时过一遍语法	第 0 章 前置基础
Lectures on Scientific Computing	Lecture 4 讲 statsmodels 回归	第 7 章 Statsmodels
Lectures on Scientific Computing (Robert Johansson)	Lecture-2-Numpy.ipynb, Notebook 形式	第 1 章 NumPy、第 2 章 SymPy、第 4 章 Pandas、第 6 章 NetworkX
MIT OCW 数值计算 (2.086)	数值方法讲义	附录
NumPy 中文学习站	中文入门与文档镜像	第 1 章 NumPy

资料	说明	用于
Python 教程-廖雪峰 (中文)	中文查漏	第 0 章 前置基础
Python 数据可视化 (知乎/博客)	中文快速入门与经验	第 5 章 Matplotlib
SciPy Lecture Notes	系统、带图, 含 SciPy 章节	第 3 章 SciPy
SciPy Lecture Notes	Matplotlib 进阶绘图讲义	第 5 章 Matplotlib
SciPy Lecture Notes -NumPy	系统、带图, 覆盖数组/广播/运算	第 1 章 NumPy
SciPy Lecture Notes (Pandas 章节)	偏科学计算, 含数据整理	第 4 章 Pandas
SciPy Lecture Notes (网络章节)	科学计算整体, 含图与网络	第 6 章 NetworkX
SciPy Lectures -Numpy / Linear algebra	数值实现视角	附录
SciPy Lectures -Optimization / Integration	优化/积分/ODE 实用教程	附录
SciPy Lectures -SymPy	系统、带例子, 覆盖符号计算核心	第 2 章 SymPy
SciPy-lectures 练习	每节配练习	第 3 章 SciPy
Scientific Python Lectures (GitHub)	Lecture-3-Scipy.ipynb, Notebook 形式	第 3 章 SciPy
ZetCode 中文教程: SymPy	中文入门, 含符号运算示例	第 2 章 SymPy
numpy.net.cn (中文学习站)	中文科学计算入门 (含 Pandas 相关)	第 4 章 Pandas
scikit-learn MOOC 习题	在线交互 Quiz + Notebook	第 8 章 scikit-learn
sklearn 中文文档 (社区译本)	中文版用户指南/API 参考, 适合初读	第 8 章 scikit-learn
华为云/CSDN NetworkX 教程	中文使用详解, 含代码	第 6 章 NetworkX
可汗学院 (中文)	微积分/统计基础	附录
廖雪峰 Git 教程 (中文)	中文入门	第 0 章 前置基础
猫头虎 SciPy 入门教程	安装/模块/用例中文入门	第 3 章 SciPy
知乎专栏: SymPy 入门与实战	中文文章, 带代码与截图	第 2 章 SymPy
知乎: 复杂网络建模 (Python+NetworkX)	复杂网络建模课程代码 (中文)	第 6 章 NetworkX
课程仓库参考 (原: Grape-book (图深度学习))	图神经网络进阶 (延伸)	第 6 章 NetworkX
课程仓库参考 (原: 开源社区 fantastic-matplotlib)	中文、渐进式 Matplotlib 教程	第 5 章 Matplotlib
课程仓库参考 (原: 开源社区 wow-plotly)	Plotly 交互式教程 (对比阅读)	第 5 章 Matplotlib
课程仓库参考 (原: 开源社区 聪明办法学 Python v2)	前置 Python 课程	第 4 章 Pandas、第 8 章 scikit-learn
课程仓库参考 (原: 社区时间序列相关笔记)	时间序列特征与规则入门	第 7 章 Statsmodels
课程仓库参考 (原: 聪明办法学 Python v2 (开源社区))	前置课程: Chap0 安装、Chap1 启航、Chap2-6 基础语法	第 0 章 前置基础
课程仓库参考 (原: 聪明办法学 Python v2)	前置课程	第 1 章 NumPy、第 2 章 SymPy、第 7 章 Statsmodels
课程仓库参考 (原: 聪明办法学 Python (前置))	前置课程	第 3 章 SciPy、第 5 章 Matplotlib

四、习题与实战

资料	说明	用于
100 Pandas Puzzles	100 道 Pandas 小题带答案	第 4 章 Pandas
Hands-On ML 练习	每章自带练习（含答案）	第 8 章 scikit-learn
NTNU: Exercises and solutions in symbolic mathematics (SymPy)	含 SymPy 符号计算练习题与答案	第 2 章 SymPy
NumPy 习题（中文版, Data-Science-Notes）	含 numpy-100 中文整理	第 1 章 NumPy
Pandas Exercises (guipsamora)	分主题练习（清洗/分组/时间序列）	第 4 章 Pandas
SciPy Cookbook	大量实战小例子（老但仍有效）	第 3 章 SciPy
numpy_exercises by 4GeeksAcademy	在线运行版本（Binder 友好）	第 1 章 NumPy

五、开源项目与配套仓库

资料	说明	用于
Python for Data Analysis 3rd (Wes McKinney)	Pandas 作者亲著，第三版配套代码	第 4 章 Pandas
Python for Data Analysis (Wes McKinney)	14 章包含 statsmodels 回归示例	第 7 章 Statsmodels
dair-ai/Mathematics-for-ML	按主题收集资源的清单	附录
matplotlib-cn（中文文档）	Matplotlib 中文文档镜像	第 5 章 Matplotlib
mint-lab/prog_meets_math	“Python Meets Math”: Calculus / Linear Algebra / Optimization / Probability + 代码	附录

六、中文资料与镜像

资料	说明	用于
清华 Anaconda/PyPI 镜像	conda/pip 都从这里加速	第 0 章 前置基础
清华 Miniconda 镜像	国内下载加速	第 0 章 前置基础
腾讯云开发者社区：Python 科学计算之 SymPy	中文案例集合	第 2 章 SymPy

七、其他

资料	说明	用于
Big-O Cheat Sheet	常用复杂度速查	附录
Optimization and Fit Cookbook	拟合示例详解	第 3 章 SciPy
Python Cheat Sheet	速查表	第 0 章 前置基础
Seaborn API	函数索引	第 5 章 Matplotlib
Seaborn 示例	多分组/分面/热力图示例	第 5 章 Matplotlib

资料	说明	用于
mml-book Part II	回归/分类/概率视角	附录

按章节快速查找

章节	直接查看该章参考
第 0 章 前置基础	references.md
第 1 章 NumPy	references.md
第 2 章 SymPy	references.md
第 3 章 SciPy	references.md
第 4 章 Pandas	references.md
第 5 章 Matplotlib	references.md
第 6 章 NetworkX	references.md
第 7 章 Statsmodels	references.md
第 8 章 scikit-learn	references.md